Cohen–Sutherland Line Clipping Algorithm

C++ program using graphics.h (BGI-compatible environment)

```
#include <graphics.h>
#include <iostream>
using namespace std;

const int INSIDE = 0;
const int LEFT   = 1;
const int RIGHT  = 2;
const int BOTTOM = 4;
const int TOP    = 8;

int computeCode(double x, double y, double xmin, double ymin,
                double xmax, double ymax) {
    int code = INSIDE;
    if (x < xmin) code |= LEFT;
    else if (x > xmax) code |= RIGHT;
    if (y < ymin) code |= BOTTOM;
    else if (y > ymax) code |= TOP;
    return code;
}

bool cohenSutherlandClip(double &x1, double &y1, double &x2, double &y2,
                         double xmin, double ymin, double xmax, double ymax) {
    int code1 = computeCode(x1, y1, xmin, ymin, xmax, ymax);
    int code2 = computeCode(x2, y2, xmin, ymin, xmax, ymax);

    while (true) {
        if ((code1 | code2) == 0) return true;
        if ((code1 & code2) != 0) return false;

        int outCode = code1 ? code1 : code2;
        double x = 0.0, y = 0.0;

        if (outCode & TOP) {
            if (y2 == y1) return false;
            x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
            y = ymax;
        } else if (outCode & BOTTOM) {
            if (y2 == y1) return false;
            x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
            y = ymin;
        } else if (outCode & RIGHT) {
            if (x2 == x1) return false;
            y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
            x = xmax;
        } else if (outCode & LEFT) {
            if (x2 == x1) return false;
            y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
            x = xmin;
        }

        if (outCode == code1) {
            x1 = x; y1 = y;
            code1 = computeCode(x1, y1, xmin, ymin, xmax, ymax);
        } else {
            x2 = x; y2 = y;
            code2 = computeCode(x2, y2, xmin, ymin, xmax, ymax);
        }
    }
}

int main() {
    double x1, y1, x2, y2;
```

```
    double xmin, ymin, xmax, ymax;

    cout << "Enter line endpoints x1 y1 x2 y2: ";
    cin >> x1 >> y1 >> x2 >> y2;
    cout << "Enter clipping window xmin ymin xmax ymax: ";
    cin >> xmin >> ymin >> xmax >> ymax;

    if (xmin >= xmax || ymin >= ymax) {
        cout << "Invalid clipping window. Ensure xmin < xmax and ymin < ymax.\\n";
        return 1;
    }

    double cx1 = x1, cy1 = y1, cx2 = x2, cy2 = y2;
    bool accepted = cohenSutherlandClip(cx1, cy1, cx2, cy2,
                                        xmin, ymin, xmax, ymax);

    cout << (accepted ? "Line accepted after clipping.\\n"
                      : "Line rejected: outside clipping window.\\n");

    initwindow(800, 600, "Cohen-Sutherland Line Clipping");

    setcolor(WHITE);
    rectangle((int)xmin, (int)ymin, (int)xmax, (int)ymax);

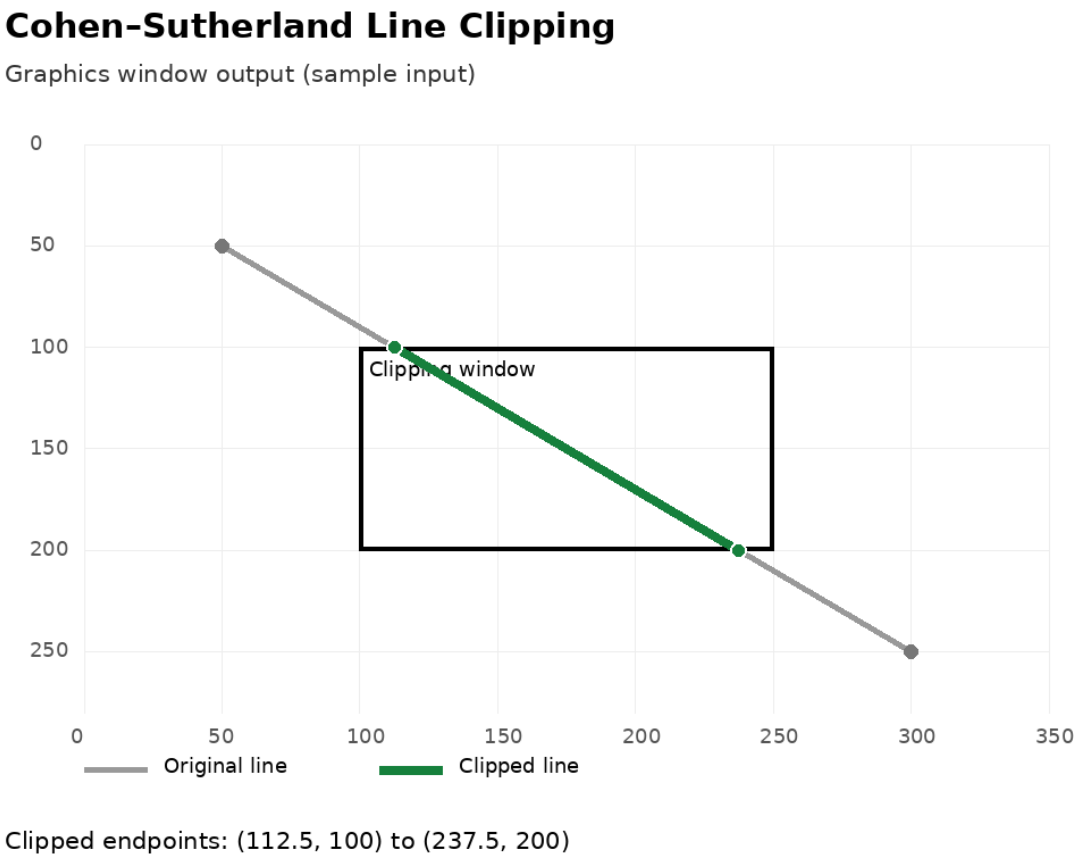
    setcolor(LIGHTGRAY);
    line((int)x1, (int)y1, (int)x2, (int)y2);

    if (accepted) {
        setcolor(GREEN);
        line((int)cx1, (int)cy1, (int)cx2, (int)cy2);
        cout << "Clipped endpoints: (" << cx1 << ", " << cy1
             << ") to (" << cx2 << ", " << cy2 << ")\\n";
    }

    getch();
    closegraph();
    return 0;
}
```

Sample Output

Input: 50 50 300 250 Clipping window: 100 100 250 200




```
Line accepted after clipping.
Clipped endpoints: (112.5, 100) to (237.5, 200)
```

Note: This is a clean print-friendly illustration of the expected graphics.h output. In the original BGI graphics window, coordinates are drawn in screen space, where y increases downward.